

## DP2 Peer Review Reflection

Github: <https://github.students.cs.ubc.ca/CPSC416-2025W-T2/mtahir03>

Implementation: 5e1ea4c4374a450edfca7bd42db64bdeaa072038 (hash)

Location: projects/dp2/lab/src

### Chosen Design:

<https://github.students.cs.ubc.ca/CPSC416-2025W-T2/jlu43/blob/main/dp2-design-document.pdf>

Original feedback: <https://github.students.cs.ubc.ca/CPSC416-2025W-T2/jlu43/issues/9>

My original feedback said the design was strong overall, but I raised three main concerns: the GC / lagging replica recovery path was not explicit enough, some ambiguous parts of the design were still underspecified despite the overall detail, and concurrent proposals / holes might be problematic if a later slot was accepted while an earlier one was still unresolved. Looking back after implementation, those were the right areas to focus on. The major issues we hit were not with the basic Paxos quorum argument, but with hole handling, leader churn, and recovery speed under unreliable conditions.

### Failure Model

The original design's stated failure model was correct. I did not discover a fundamentally different model, but implementation did show that one aspect needed more emphasis: the interaction between partitions, leader instability, and holes. That is how can we ensure liveness and progress when multiple error conditions (ie. unreliable messages, partitions) overlap. To be clear, this was not a result of any clear error in the design, my critique in this section is mainly focused on the idea that vagueness/certain design choices resulted in slow progress.

For example, the design already says that liveness depends on partial synchrony, majority connectivity, and leader stability, and that dueling leaders slow progress but do not violate safety. However, some of our tests failed during constant repartitioning, the issue was not that the protocol became unsafe; it was that repeated preemption and unstable leadership made progress too slow. Subtle choices such as heartbeat timer lengths, or the messages we send and when we send them to ensure catchup affect both the recovery time and also the maximum allowable message threshold.

Additionally, my original feedback also asked for more clarity on lagging follower recovery. The design did specify the intended mechanism: followers send followerSlotOut in heartbeat replies, the leader computes a GC watermark, and the leader sends missing Decisions to lagging followers. So the issue was not that recovery was omitted. The issue was that this design did not maintain significant progress during periods of highly unreliable messaging (ie. the heartbeats are not received) or during partitioning. Since execution is in-order, a follower can have later

chosen slots but still be blocked at the first missing earlier slot until that Decision replay happens. That became very visible when debugging progress after healing a partition.

## **Protocol**

I did not discover missing roles or missing quorum requirements.

The main protocol gap was around holes and slot management. The design explicitly allows concurrent slots, says the leader keeps filling slots with pending commands, and also notes that if  $\text{slot\_out} < \text{slot\_in}$  and  $\text{log}[\text{slot\_out}]$  is undecided, there is a hole blocking execution and the leader should be proposing for that slot. What it did not specify clearly enough was the actual slot-allocation policy once both conditions are true: there is an earlier hole, but the leader also has new client work to append. For example from the design we see “slot\_in” as being the next place to propose and it is equal to the highest-proposal + 1. However, this breaks when we have concurrent proposals. Consider a situation in which we have proposed slot 10, but slot 7 is also missing because of lost messages. What should slot\_in point to? When we used a more aggressive slotIn-style allocation strategy, later proposals moved forward while an earlier hole still blocked slotOut, which caused a “Progress after partition healed” timeout.

A second protocol gap was step-down behavior under preemption. The design correctly says that when preempted, the leader sets `isActive = false`, updates its ballot, and waits for heartbeat-based re-election rather than immediately restarting Phase 1. What was unspecified is that local leader-only state must be cleared when this happens. In our implementation, stale proposal / phase-2 state surviving step-down contributed to repartitioning slowness because old in-flight state kept interfering with fresh proposals.

## **Observability**

I think I would also explicitly log what state is cleared on every step-down path. The design says to record leader transitions, but the repartitioning failures showed that it would also be useful to know whether stale proposal / phase-2 state survived those transitions.

## **Original Feedback Critique**

The main thing I would improve in my original feedback is that I should have pushed harder on the **interaction** between these mechanisms. I talked about GC recovery and holes separately, but the real implementation pain came from their combination with leader churn and retries. In future design reviews I would ask more direct questions like: if a follower misses an earlier decision and later slots are already chosen, what exact mechanism guarantees that the earliest hole is repaired quickly enough for slotOut to keep moving under partitions and re-elections?

Again, to be clear, I still think the original design was good. The critiques here are inherently difficult to consider in a design without making it overly verbose/specific and encroaching upon what may be responsibilities to be decided in implementation.